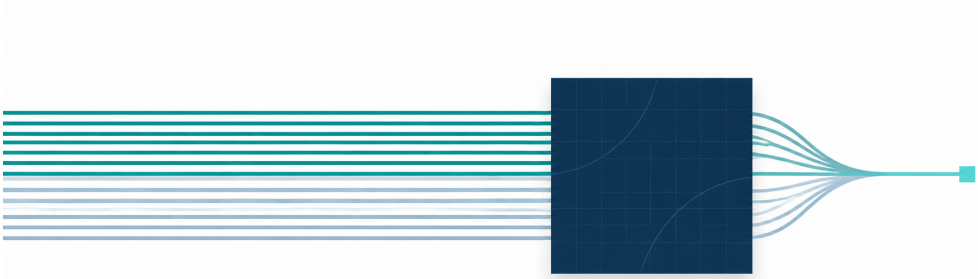
香山昆明湖 V2

自定义向量点积指令

第三阶段协同仿真与性能分析

技术报告



工程组合： 0f ＋ firstdot V2 ＋原 C8

团队名称： 待团队确认

成员： 待团队确认

学校： 待团队确认

材料日期： 2026 年 11 月 2 日

文档修订： 2026 年 11 月 5 日

目录

第三阶段：协同仿真与性能分析 1

3.1 NEMU 扩展与协同仿真 1

3.1.1 独立参考模型与架构行为 1
3.1.2 指令译码与异常处理 2
3.1.3 提交边界上的差分比较 3
3.1.4 高位注错与正常参考对照 4

3.2 RVV 对照与周期测量方法 5

3.2.1 等价 RVV 指令序列 5
3.2.2 计数窗口与实际执行边界 6
3.2.3 工作负载、输入与重复测量 8
3.2.4 加速比与统计方法 9

3.3 点积与矩阵向量计算性能 10

3.3.1 四类点积微基准 10
3.3.2 单次调用与独立批量 11
3.3.3 依赖链与流式循环 12
3.3.4 每次点积的平均成本 12
3.3.5 GEMV 调用与配置复用收益 13
3.3.6 硬件与软件收益的对应关系 13

3.4 普通 C 接口与软件包接入 14

3.4.1 firstdot 与 C8 的调用组织 14
3.4.2 输入输出布局与调用约定 15
3.4.3 编译期选择与 RVV 回退 16
3.4.4 对象、装载镜像与调用表核对 16
3.4.5 接入检查器的正负控制 17
3.4.6 接入结果与功能记录复用 19

3.5 面积、时序与实现成本 20

3.5.1 评估条件与主要结果 20
3.5.2 寄存状态与综合面积 20

3.5.3 时序约束与路径分类 21
3.5.4 父级关键路径与改进方向 22
3.5.5 取消关钟与成本结果 23
3.5.6 门控修复的工程取舍 23

3.6 验证汇总与设计结论 23
3.6.1 功能、复现与性能检查汇总 23
3.6.2 优化候选的实测取舍 26
3.6.3 阶段结论 27

3.7 工程复现资料 27
3.7.1 固定版本与主要运行产物 27
3.7.2 两槽结构对照的样本记录 28
3.7.3 硬件对照与工具身份 28
3.7.4 离线复算与模型数据 29
3.7.5 取消关钟修复的对照输入 29
3.7.6 修复前后逐窗周期核对 30
3.7.7 综合面积与完整接口 31
3.7.8 五档时序详细结果 31
3.7.9 复算入口与实验归档 32
3.7.10 C 接口与编译开关 33
3.7.11 对象构建与接入命令 33
3.7.12 链接对象与检查入口 34
3.7.13 宿主运行记录与并发审计 34
3.7.14 工程使用与回退资料 35
3.7.15 技术资料索引 35

第三阶段：协同仿真与性能分析

工程组合：0f ＋ firstdot V2 ＋原 C8　材料日期：2026 年 11 月 2 日

本阶段完成 NEMU 指令扩展与 RTL 协同仿真，使用现有 RVV 指令构造等价对照，并评估点积及矩阵向量计算的周期与实现成本。最终硬件版本记为 0f，包含两槽弹性执行结构（O2）、低 32 位结果保存（H1）和 DIV 取消关钟修复。软件中的 firstdot 表示首段点积初始化方法，C8 表示 8×32 调用中的向量配置复用入口。

3.1 NEMU 扩展与协同仿真

3.1.1 独立参考模型与架构行为

VDot 的硬件实现和 NEMU 参考模型共享同一份指令语义。输入是两个向量寄存器低 64 位中的八对有符号字节，输出是八项乘积之和：低 32 位保存二补码结果，高 96 位清零。硬件通过并行乘法、归约和两槽控制产生响应，NEMU 则按软件顺序提取元素并求和。两种实现采用不同的计算组织，使比较同时覆盖算术结果和结果布局。

参考模型的取数过程从架构寄存器开始。每个字节先解释为 signed int8，再扩展到能够容纳乘积与总和的有符号类型；求和初值为零，不读取目的寄存器的旧值。读取两个源的快照后再写回目的寄存器，因此 vd 与 vs1、vs2 重合时仍使用同一条指令执行前的源数据。这样的步骤与软件可见语义直接对应，也便于检查 C 语言整数提升、符号扩展和移位处理是否符合预期。

参考侧使用完整 128 位向量值承载结果。对负数而言，符号只保存在低 32 位二补码中，高 96 位仍为零；这是本指令约定的结果格式。若把 32 位结果符号扩展到整个向量，低位算术可能正确，高位比较仍会立即暴露布局错误。硬件内部采用 32 位结果寄存器后，在公开输出处补零，正是利用了这一固定的架构属性。

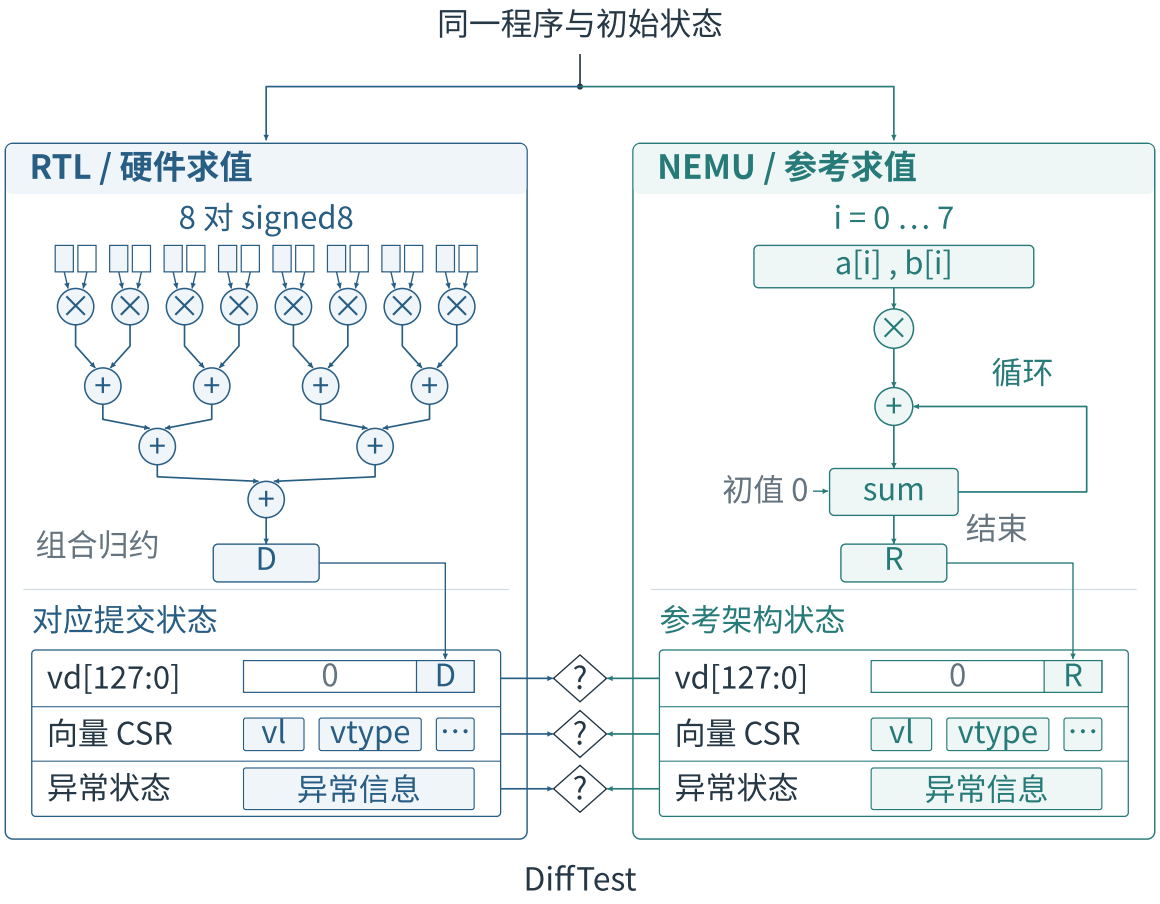


图 3-1 RTL 与 NEMU 独立模型的 DiffTest 框架

相同程序与初始状态分别进入 RTL 和 NEMU。左侧八路乘法和组合归约、右侧 C 循环仅表示独立求值结构，树层数不是流水级数，循环次数不是 DUT 周期。算术结果写入各自向量状态，CSR 与异常来自模型整体；DiffTest 按对应架构提交比较完整结果及状态。D、R 仅为两端结果标签，FU 响应不等于 ROB 提交。

3.1.2 指令译码与异常处理

NEMU 先按 custom-0 指令字匹配项目编码，再检查向量配置。正常路径使用 SEW8、LMUL1、VL8、vstart0 和 VLEN128；非法配置进入异常处理，目的向量保持原值。译码与执行检查共同决定指令能否生效，因此测试既覆盖操作码和功能字段，也覆盖同一合法机器字在不同向量状态下的行为。

香山固定版本中，译码阶段识别的非法指令与功能单元发现的 VL 非法具有不同的 xTVAL 表现。平台适配按对应的指令和架构状态描述这些条件，并比较异常码、陷阱 PC、xTVAL 及目的状态。报告中的特权执行记录来自 M 模式。该平台规则集中保存在 reference 配置中，正常算术路径仍由独立求和得到结果。

异常测试中的初始状态与正常测试同样重要。将目的向量设置为可辨认的非零值，能观察非法操作是否错误清零或部分更新目的；源寄存器和 CSR 快照则用于核对异常前后的状态变化。把这些观察与异常 PC 对应起来，可以区分译码错误、执行阶段非法配置和异常后的状态恢复问题。

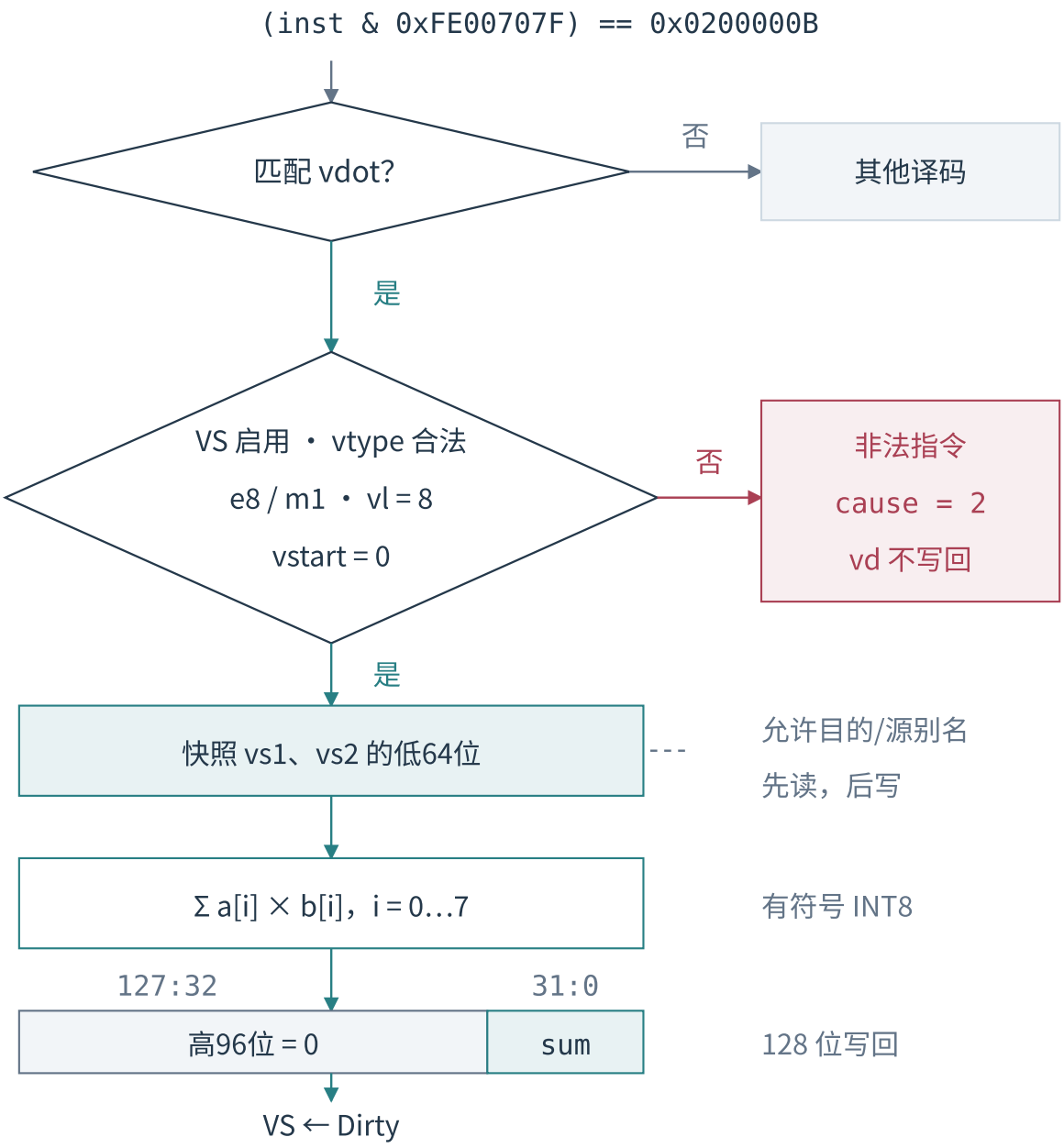


图 3-2 NEMU 中 vdot 的识别、状态检查与独立执行

精确匹配 vdot 编码后，所有向量状态条件均须满足；vtype 合法包括 vill 与保留位检查。合法路径先快照两源，再求八项有符号 INT8 乘积之和，完整写回 128 位（高 96 位为零、低 32 位为和的补码）并将 VS 置脏。其他译码与状态非法分支分别处理。NEMU a48a11f、补丁 1.1.0；xTVAL 沿用固定平台策略，已验证范围为 M 模式。

3.1.3 提交边界上的差分比较

DiffTest 在架构提交边界比较 DUT 与参考模型。前端取指、功能单元响应和 ROB 提交分别代表不同的进度，比较器以最终提交的指令与架构状态建立对应关系。对 VDot 而言，关键内容包括机器指令、目的寄存器、完整向量结果及向量 CSR；对异常指令，还包括陷阱入口和相关异常字段。

运行记录按启动、指令执行、差分检查、程序结束四个阶段组织。正常用例同时检

查计划中的测试数量、完整结果、GOOD TRAP 和进程退出状态。这样可以把程序自行报告的功能结果与外部差分检查联系起来：前者核对测试期望，后者核对 DUT 提交状态与独立模型的一致性。保留 stdout、stderr 和退出码，也便于定位构建、装载和目标程序执行之间的故障位置。

不同层次的测试采用各自的计数单位。主机模型以输入向量和测试方法计数，局部 RTL 以事务与控制场景计数，全核以程序、用例和窗口计数。R5 全核功能矩阵的 117 个不同用例与 124 次分片执行分别描述用例覆盖和实际执行安排；新控制诊断中的 131072 字节则描述被核对的寄存器结果量。把这些单位列清楚，能够准确反映每一层验证所完成的工作。

点积测试检查整个目的向量，同时记录其他向量寄存器与 CSR。VDot 路径仅更新约定的目的寄存器；RVV 等价路径需要中间向量寄存器，检查器据预先声明的 scratch 集合判断这些变化。由此既能检查计算结果，又能区分允许的临时状态与意外破坏。

源与目的别名是测试的重要部分。先读取两个源、再清理或覆盖目的，能够保证 vd 与任一源重合时仍计算原来的八对输入。RVV 序列也遵循相同的数据顺序：先生成宽化乘积，再清零目的并归约。若先清目的寄存器，别名场景中的源数据会被破坏，即使无别名样例全部通过也会留下语义缺口。

访存测试还检查输出槽和前后护栏。每个结果使用 16 字节槽保存完整向量，连续结果之间的步长与实际存储宽度一致。逐槽比较能够发现结果正确但写入地址错误、前一槽覆盖后一槽或高位残留等问题。输入高半区的非零哨兵则检查实现是否意外使用了规定输入范围之外的数据。

3.1.4 高位注错与正常参考对照

bit100 负控针对向量结果的高位比较。合法 VDot 结果的高 96 位均为零，选择完整向量的 bit100 注错，相当于改变高 64 位中的 bit36。低 32 位算术结果可以保持一致，而高位差异应被 DiffTest 检出，因此这项测试直接检查完整向量比较是否生效。

图中的两组记录来自 2026 年 10 月 24 日 core-01 实验。负控运行加载故障注入参考库，在 PC 0x80001bdc、指令字 0x0294050b、目的寄存器 v10 处出现首次失配，以 exit 1 结束；正常对照使用同一测试镜像和正常 reference，完成 15 个用例并以 GOOD TRAP、exit 0 结束。两次独立运行的镜像一致，使参考库成为对照中的关键变化项。

首次架构失配 PC 定位状态差异的起点，终止事件记录模拟器的退出位置。机器字、寄存器和高位差异共同对应注错目标；正常对照确认同一镜像、启动流程和数据能够完成整组测试。

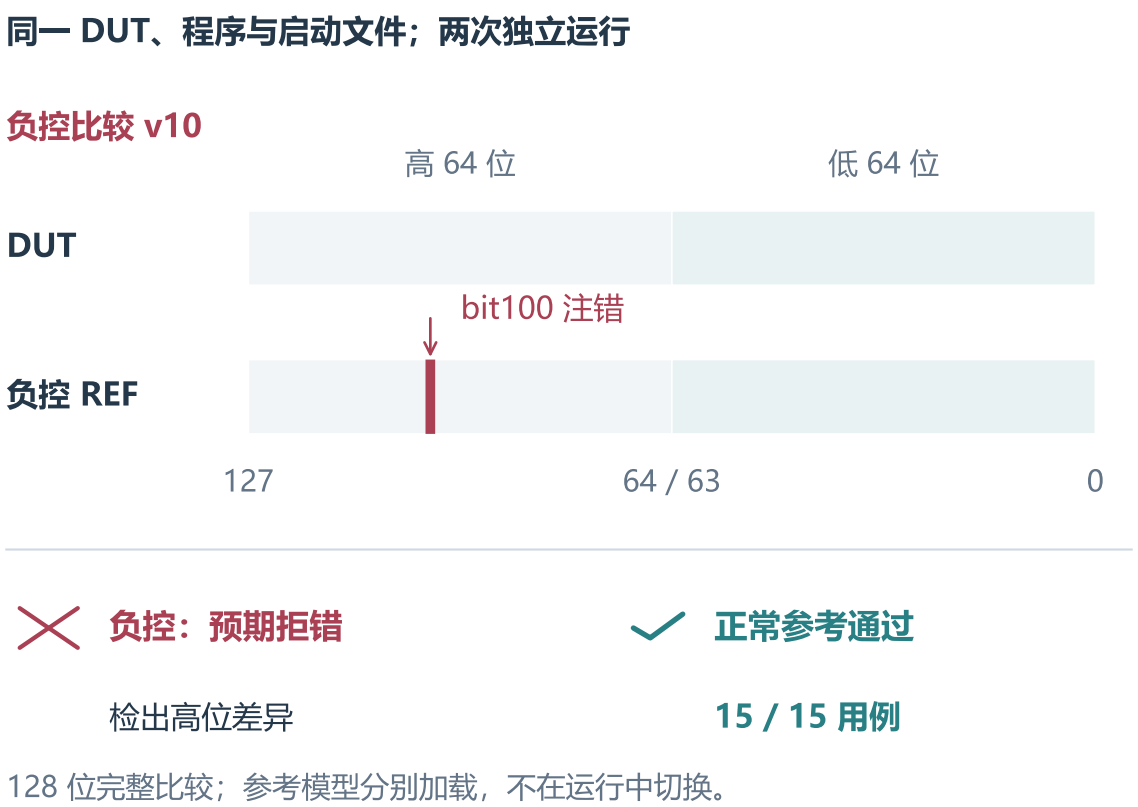


图 3-3　bit100 负控与正常参考对照

负控在 v10 的 bit100 检出差异，首次失配 PC 为 0x80001bdc；正常参考对照完成 15 个用例并以 GOOD TRAP、exit 0 结束。

3.2　RVV 对照与周期测量方法

3.2.1　等价 RVV 指令序列

两条路径均将低 8 个 signed int8 元素分别相乘并求和，目的向量低 32 位保存二补码结果，高 96 位清零。检查覆盖完整 128 位及寄存器副作用：vdot 保持其他向量寄存器，RVV 的中间操作使用预先声明的 scratch 集合。

表 3-1　等价 RVV 指令序列

RVV 路径	点积主体指令数	向量 scratch	入口与出口
已知配置单次	5	v2、v3	e8/m1/VL8、tu/mu；退出恢复相同配置
通用配置单次	7	v2、v3，另用 t3/t4 保存 CSR	保留原 VL/vtype，测试四种 tail/mask policy

表 3-1 等价 RVV 指令序列（续）

RVV 路径	点积主体指令数	向量 scratch	入口与出口
独立四路批处理	14 / 4 个点积	v16–v23	e8/m1/VL8、tu/mu；退出恢复相同配置

已知配置序列：vwmul.vv → 设置 e16/m1/VL8 → vmv.v.i vd,0 → vwredsum.vs → 恢复 e8/m1/VL8。e16 下 8 个元素正好清零整个目的寄存器；宽化归约使用 tu，保留已经清零的高位。序列先乘后清目的寄存器，从而支持源与目的别名。通用序列增加 csrr vl/vtype，末尾用 vsetvl 恢复。四路批处理共享两次 vtype 设置，充分利用批量调用的配置复用。

每次调用保存并比较全部 32 个向量寄存器，按声明排除 RVV scratch；检查 VL、vtype、vstart、vxrm、vxsat、vcsr、实际循环次数以及缓冲区前后 canary。访存组逐个比较全部 128 位输出，并检查未使用输出空间。C 判定器使用真实源文件进行原生及 UBSan 位翻转测试。

3.2.2 计数窗口与实际执行边界

固定版本的 DecodeUnit.scala 为普通 CSR 设置 waitForward/blockBackward，而 Rename.scala 会解除大部分只读 CSR 的这些限制。初始观察中，cycle 与 instret 指令在 ROB 仍有旧指令时进入，相关标志为零。计时实验因此通过实际分派、采样和退休记录确定窗口顺序。

软件测量边界加入 fence rw,rw。该香山版本的 fence 通过等待旧指令、约束后续分派和 flush 形成边界，日志记录其与 CSR 及主体指令的执行关系。窗口定义依赖这套固定微架构实现，生产核的执行规则保持原样。

窗口结构如下（所有实验分支相同）：

```
fence; START; fence
read instret_begin; fence
read cycle_begin; fence
执行显式汇编主体（含循环控制）
fence; read cycle_end; fence
read instret_end; fence
STOP; fence
保存结果；完整状态检查；打印
```

观察器由 VDOT_T13_TRACE=1 启用，记录 CSR 实际采样、计数器入 ROB 及窗口内退休信息，并与 T12 观察器一起构建。它提供只读事件观察，数据通路、复位、忙闲接口和 NEMU 计时维持原实现。

窗口检查包含以下六项：

1. 精确匹配四条计数器指令的 PC、机器码、ROB 身份与顺序。

2. CSR 读值与对应退休写回数据相等，采样位于入队和退休之间。

3. 每条计数器的前一 fence 已退休，后一 fence 在计数器退休后才退休。

4. cycle 差值等于硬件全核采样 tick 差值。

5. instret 差值等于真实退休计数，也等于从 ELF 机器码展开有限循环得到的指令数。

6. 完整窗口及 UART 记录数量、顺序与 manifest 一致；真实 exit0、DiffTest 启用、完整成功 summary 后 GOOD TRAP。

绝对计数器可能大于 2^53，解析和差值计算全程使用 Python 整数。FU 子时钟受门控控制，局部事件先通过 T12 记录的父级 VFEX4 accept 逐事务关联，再与 ROB 事件比较；全核窗口周期使用统一的全局计数。

校准场景包括空窗口、8 条整数指令和 32 条依赖除法。正式结果保留完整的屏障、取指和循环开销；空窗口用于观察固定成本，小工作量场景则结合这些成本解释周期。

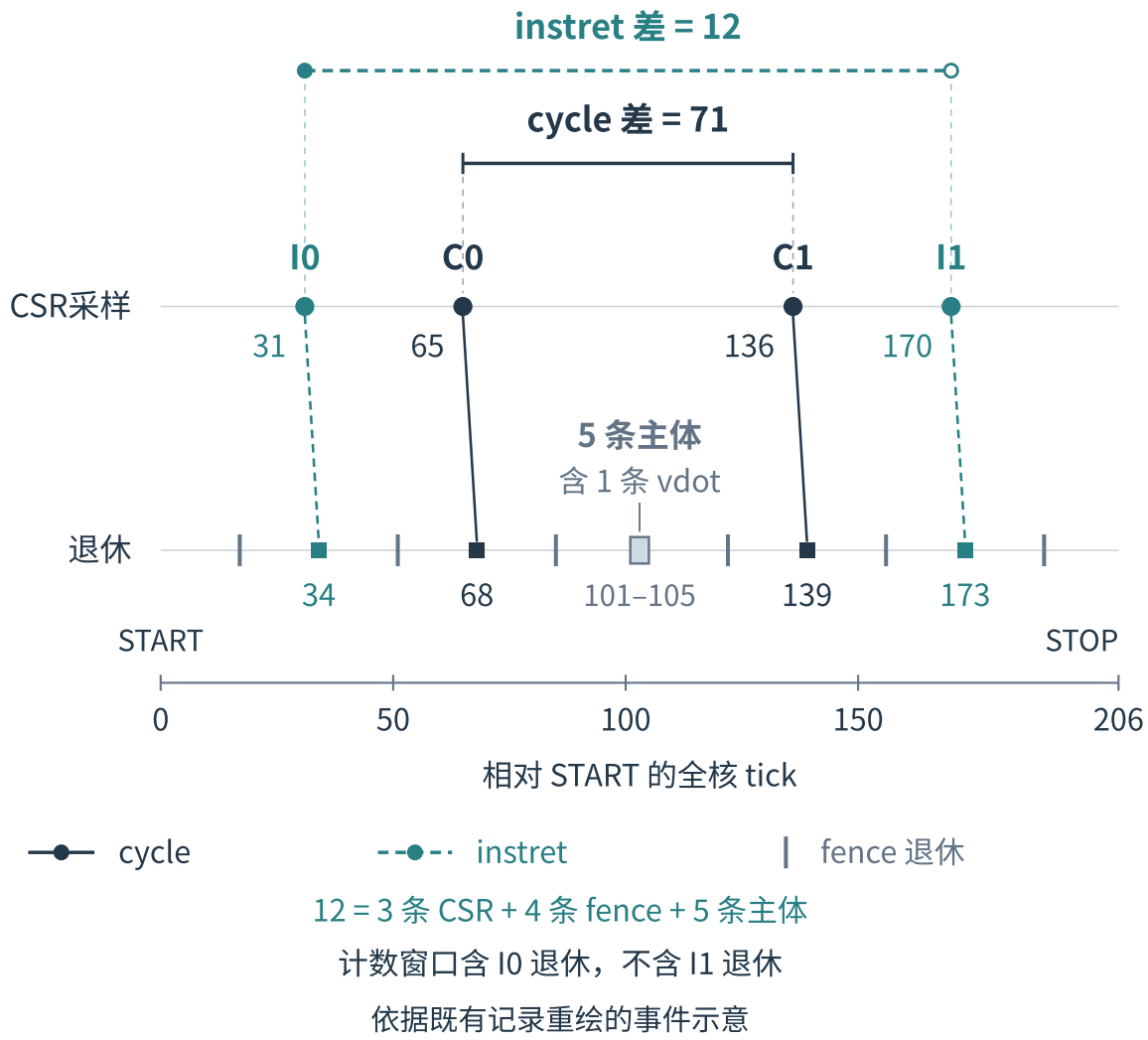


图 3-4　计时窗口 CSR 与退休边界

四条 CSR 分别记录 cycle 与 instret 起止值；图中标出采样和退休位置。固定 0f 的 after/dataset0/replay0/seq4 正式样本 repeat=2，cycle 差为 71、instret 差为 12。周期含隔离与循环开销，不扣空窗口；START/STOP 标记跨度为 206 个全核 tick。

3.2.3　工作负载、输入与重复测量

表 3-2　工作负载、输入与重复测量

场景	n	点积数	含义
single	1	1	已知配置完整调用窗口
chain	16、64、256	n	目的向量作为下一次源，观察依赖限制

表 3-2 工作负载、输入与重复测量（续）

场景	n	点积数	含义
batch	1、8、32	4n	每轮四个独立目的，比较共享配置的 RVV 批处理
stream	1、16、64	n	两个 128 位加载、点积、128 位存储以及指针更新
generic	1、16、64	n	原 vtype=192 的 CSR 保留对照

链式实验将上一结果的完整向量作为下一次源，数值可能逐步趋零，寄存器依赖关系则持续存在。批处理使用四个独立目的并共享输入和配置，用于观察独立请求吞吐。两者分别突出反馈等待与连续接收，对应不同的软件组织方式。

三组输入包括正极值、负极值和混合符号；高半区设置非零哨兵，访存组加入固定种子的随机数据。功能矩阵使用原始 22 组定向和前 32 组随机数据，轮换七种别名与四种 policy；54 对数据覆盖选定的别名和策略组合。另用固定 NEMU 对全部 10,022 组数据重新检查精简基线，明确与全核执行层分开计数。

每个配置在同一镜像内运行 6 轮配对，按执行轮次保留首次样本、预热和 4 轮正式样本；顺序按 AB/BA 交替，缓存状态随实际运行演进。每种配置单独分片，两个实现的入口均 64 字节对齐；代码长度、地址和寄存器压力的自然差异保留并记录。

每个性能镜像以 seed=1 在新的 EMU 进程中运行三次，按相同 seq 比较周期和退休指令数，完成确定性重放核查。重放记录与正式统计样本分别保存。原测量批次按最多 8 个宿主进程安排，各 DUT 均采用相同单核和 EMU_THREADS=1 配置；加速比使用 DUT 周期，宿主秒数用于记录执行成本。

3.2.4 加速比与统计方法

首次、预热和正式窗口完整保留。每个配置与数据集分别汇总原始周期、周期/点积、退休指令数及正式样本 min/median/max；配对加速比为 RVV cycles / vdot cycles，周期减少比例为 1 - vdot / RVV。配对保持场景、n 和数据集一致，同时列出加速和退化结果。

结果分为局部 FU 响应、完整序列窗口、独立批量吞吐和包含访存的循环四个层次。历史单在途 FU 的 E3/II4 描述局部接口行为；完整序列窗口则包含相同的隔离与循环开销。将各层指标对应到实际工作量，可以分别解释硬件执行和软件调用成本。

本节使用周期指标比较固定工作负载的执行效率。面积与静态时序由后续成本实验

独立测量，C8 软件收益则来自固定 GEMV 调用矩阵。三类结果分别呈现硬件周期、物理实现成本和应用调用效果。

正式加速比使用两种方法的总周期之比。设第 i 个配对窗口的 RVV 周期为 A_i、VDot 周期为 B_i，则汇总结果为 $\frac{\sum_i A_i}{\sum_i B_i}$。它等于各窗口加速比 $\frac{A_i}{B_i}$ 按 B_i 加权后的平均值。这样的定义对应完成整组固定工作量时两种实现各自付出的总周期，较长的窗口按其实际执行成本获得相应权重。

逐窗加速比的简单平均回答的是另一种问题：每个窗口无论长短均占相同权重。两者在样本周期不同的情况下可能给出不同数值，因此本报告沿用原测量合同的总周期比，并在样本文件中保留每一对原值。三组数据、每组四次正式窗口的数量一致，配对身份则保证统计中的工作量一一对应。

对于 0f 修复前后比较，96 对正式窗口的每对差值均为零。这比两组均值在显示精度内相同提供了更具体的信息：相同镜像和序号的窗口逐项保持周期，同时退休数也一致。面积与时序发生变化时，仍可据此把所观察到的代价定位到综合映射与路径，而不是该组软件窗口的执行周期。

3.3 点积与矩阵向量计算性能

3.3.1 四类点积微基准

下表来自 0f 同镜像对照合同，均为正式窗口均值。

表 3-3 四类点积微基准

场景	vdot cycles	RVV cycles	加速比
单次点积	71.000	85.250	1.200704×
依赖链 256	1857.000	5440.500	2.929725×
四路独立 batch128	221.000	678.000	3.067873×
加载/计算/存储 stream64	1127.500	1133.000	1.004878×

独立批量可以利用 O2 的连续接收能力，依赖链受到前后数据依赖约束，stream 场景还承担访存及循环开销。结果显示收益随工作负载改变。

O1→O2 的独立同镜像比较中，batch128 由 362.5 降至 221 周期，减少 39.0345%。H1 及 0f 修复分别保持各自 96 对正式窗口周期相同。阶段间比较采用各自冻结合同。

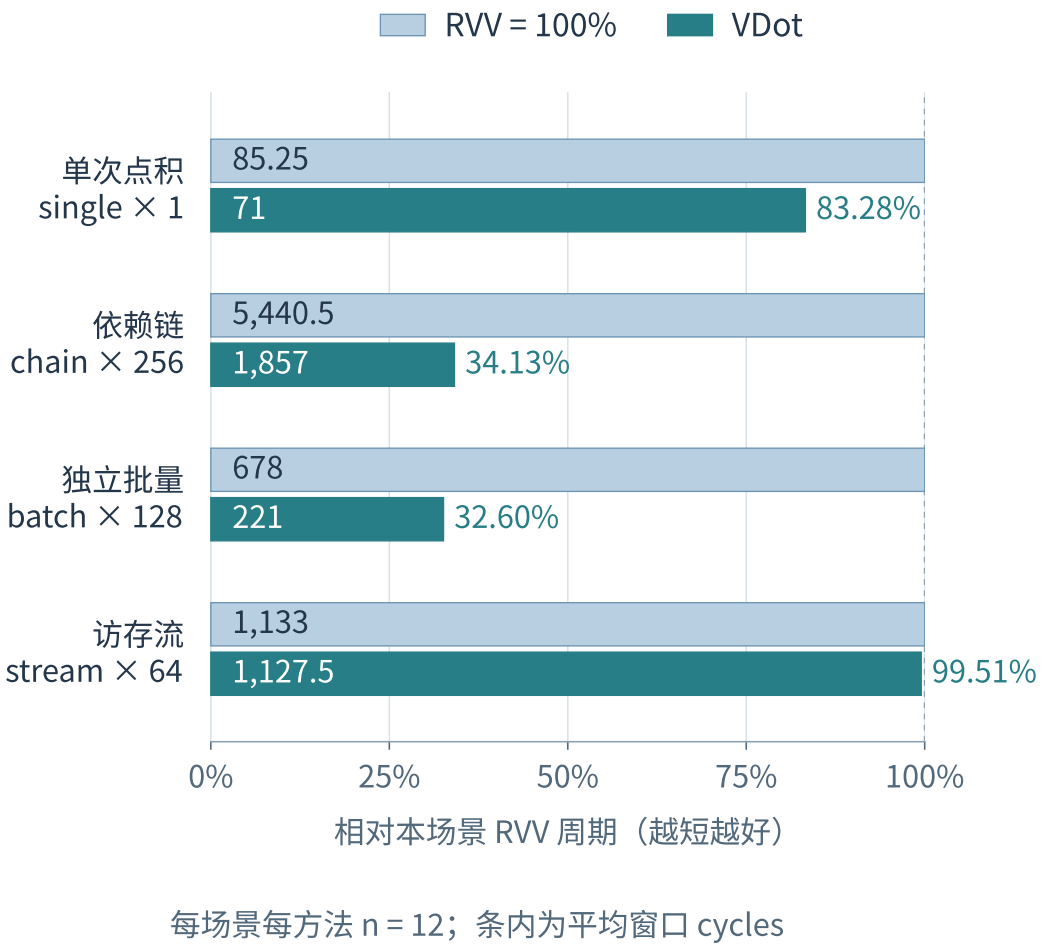


图 3-5　四类微基准的相对周期对照

取固定 after 记录，各场景、各方法均为 12 个正式样本。各场景按自身 RVV 平均窗口周期归一化为 100%，条内为实际平均窗口 cycles，条外为 VDot 周期占比，越短越好。single、chain、batch、stream 每窗口分别包含 1、256、128、64 次点积；batch 为四路独立任务。

3.3.2　单次调用与独立批量

单次点积中，VDot 为 71 周期，RVV 为 85.25 周期，加速比约 1.201。该窗口的点积工作量较小，配置、循环和测量边界占有明显比例。VDot 把多条向量算术操作收敛为一条自定义操作，节省的指令工作与共同固定开销一起形成了这一结果。

四路独立 batch128 的 VDot 均值为 221 周期，RVV 为 678 周期，加速比约 3.068。四个轮换目的为连续请求提供独立性，P 与 R 两槽可以在同一边沿完成旧 R 响应、旧 P 迁移和新请求接收。窗口中的 128 次点积与循环控制共同构成总工作量，持续接收能力在这一场景中得到更充分的利用。

O1 到 O2 的同镜像比较进一步隔离了流水组织的影响：batch128 由 362.5 降到 221 周期。O2 保留两槽状态并实现弹性推进，使前后事务能够重叠；当下游 ready 持续有效时，独立请求可逐拍接收。这个机制与实测周期变化相互对应，说明了优化为何对独立批量有效。

3.3.3 依赖链与流式循环

chain256 的 VDot 为 1857 周期，RVV 为 5440.5 周期，加速比约 2.930。上一条结果写回后，下一条才能获得所需源数据，因而该场景受到完整反馈路径的约束。发射等待、寄存器读取、功能单元执行和写回共同决定下一次请求何时可发出，局部两拍响应只是其中一段。

stream64 的 VDot 为 1127.5 周期，RVV 为 1133 周期，加速比约 1.005。两者差异较小，说明在这个加载、计算、存储和循环控制共同组成的窗口里，点积主体的优化只转化为有限的整窗收益。现有记录给出总体周期；进一步归因时，可以沿访存等待、数据依赖和单元占用逐项增加观察。

这四类结果共同描述 VDot 的使用特点：单次调用体现固定开销，独立批量体现连续接收，依赖链体现结果反馈，stream 体现数据搬运与计算的组合。应用选择可以据此关注自身工作组成，例如能否把多个点积组织为独立批量、配置是否能够跨调用共享，以及结果是否立即成为下一次输入。

3.3.4 每次点积的平均成本

在工作量明确时，总周期还可以换算为每次点积的窗口平均成本。single、chain256、batch128 和 stream64 分别完成 1、256、128 和 64 次点积，因而可以直接用对应均值除以点积数。这个指标把不同长度的循环放到相同工作单位下，便于观察批量组织和依赖关系的影响。

表 3-4 每次点积的平均成本

场景	点积次数	VDot 窗口周期/点积	RVV 窗口周期/点积
single	1	71.0000	85.2500
chain256	256	7.2539	21.2520
batch128	128	1.7266	5.2969
stream64	64	17.6172	17.7031

表中数值由前述窗口均值计算，保留四位小数。batch128 的 1.7266 周期/点积包含循环和计时边界，与功能单元逐拍接收的 II=1 共同描述系统的两个层次：接口可以每拍接受独立请求，软件窗口仍需要完成发射、循环及前后准备。两者之间的差异为进一步分析提供入口，也说明提高功能单元吞吐后还需考虑整条执行路径。

chain 与 batch 使用不同的依赖组织。chain 每次等待上一结果，batch 让四个目的独立，因此归一化后的差距体现完整反馈与连续工作方式的差异。stream 额外搬运输入和输出，其每次点积成本也包含访存与寻址。对实际软件进行选择时，先判断工作属于哪一种组织方式，再使用相应指标作为参考。

3.3.5 GEMV 调用与配置复用收益

firstdot 以首段点积初始化累加状态，后续段按约定求和。C8 在已验证 8×32 路径中复用配置，提供普通 C 显式入口和 RVV 回退。输入为对齐的 288 字节 A||x，输出为 8 个 16 字节完整槽。

表 3-5 GEMV 调用与配置复用收益

同合同 8×32 实现	完整调用 cycles
firstdot	282.25
原 C8 配置复用 vdot	273.00
本批最快合格 RVV	299.00

C8 相对 firstdot 减少 3.2772%，相对同批最快 RVV 加速 1.095238 倍。三数据集完整矩阵给出周期比较，普通 C 调用、软件包与最终镜像检查验证了 ABI、回退和接入方式。

原合格镜像、复制对照、SDK 接入版 BIN 均为 31440 字节，SHA 相同。PT_LOAD、48 条调用表项与 12 组输入数据一致，ELF 差异位于调试节。627 项输入与封存材料的身份核对，将 SDK 接入结果与原功能记录对应起来。

4×8 和 16×64 采用原 firstdot 入口，4×13 选择更快的 RVV。各尺寸沿用对应的已验证调用入口。

3.3.6 硬件与软件收益的对应关系

O2、H1 和 0f 分别解决不同问题。O2 调整两槽弹性控制，提高独立请求的重叠程度；H1 利用高 96 位恒零的语义，将结果状态压缩为低 32 位保存；0f 处理 DIV 取消后的关钟清理。把这三类改动分开，有助于分别观察吞吐、状态成本和父级控制正确性。

0f 修复前后的 192 个正式窗口组成 96 对，逐对周期差均为零，退休指令与工作量一致。四次额外 batch 重放也与对应记录相同。因此该修复在这些微基准中保持原有性能，同时解决取消后持续产生无效子时钟脉冲的问题。其面积和时序影响另由完整 VFEX4 父级综合结果评估。

firstdot 与 C8 属于软件调用层的优化。它们使用既定 VDot 语义，通过累加初始化、配置复用和调用组织减少软件开销。软件收益来自完整调用窗口的比较，硬件收益来自相应硬件版本的同镜像比较；这两条分析线最后汇合到选定的工程组合 0f ＋ firstdot V2 ＋原 C8。

3.4 普通 C 接口与软件包接入

3.4.1 firstdot 与 C8 的调用组织

firstdot 用首段点积初始化累加值，后续段按约定归约，避免在首段之前额外构造等价的初始化工作。C8 针对 8×32 路径复用向量配置，把已验证的汇编函数接到普通 C 入口。两者均保留完整输出格式和相应的 RVV 对照。

在同批 8×32 完整调用窗口中，firstdot 为 282.25 周期，C8 为 273 周期，减少 3.2772%；最快合格 RVV 为 299 周期，C8 相对其为 1.095238 倍。这个比较同时包含函数执行和该合同规定的调用成本，反映了配置复用在固定 GEMV 尺寸上的效果。

不同尺寸采用各自的已验证入口。4×8 和 16×64 保持原 firstdot 路径，4×13 选用较快的 RVV，8×32 使用原 C8。这样的选择利用了实测差异：自定义点积与配置复用对某些形状有效，而较小或不规则的工作量可能更适合现有 RVV 序列。最终软件入口据此保留明确的选择与回退方式。

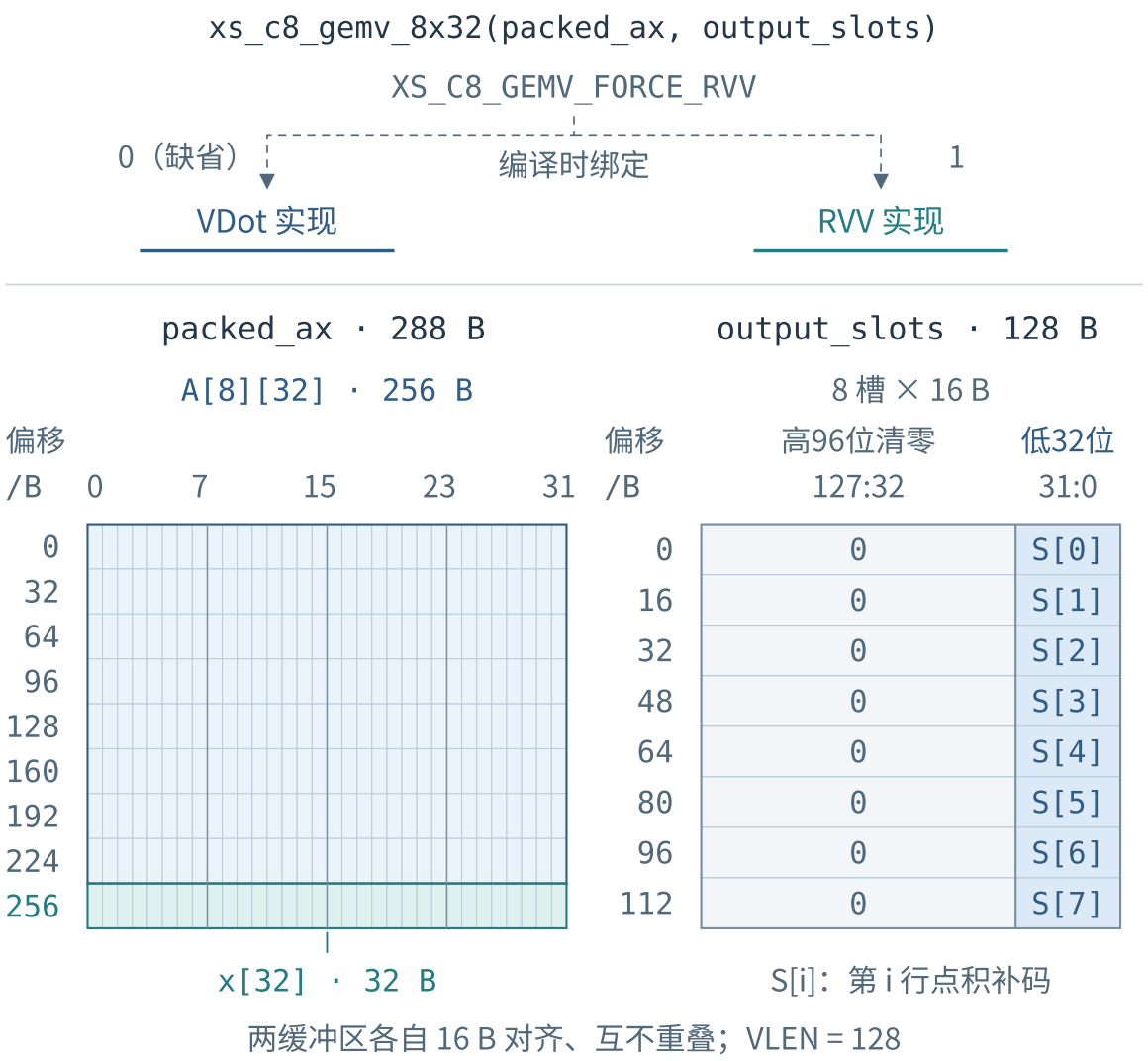


图 3-6　C8 公共调用、编译选择与输入输出布局

xs_c8_gemv_8x32 在编译时绑定 VDot 或 RVV 实现。packed_ax 包含行优先 A[8][32] 的 256 字节及其后的 x[32]，共 288 字节；output_slots 包含 8 个 16 字节槽，每槽高 96 位清零、低 32 位为点积补码。两缓冲区各自 16 字节对齐且互不重叠。横向结果位域为高位到低位示意，不表示字节端序；VLEN=128 及向量状态要求见正文。

3.4.2　输入输出布局与调用约定

函数原型为 void entry(const int8_t *packed_ax, void *output_slots)。已验证运行环境为当前香山昆明湖 V2/Nexus-AM 裸机 RV64、lp64d，**VLEN 恰好 128（vlenb=16）**，VS 已启用。默认路径要求该硬件实现项目的自定义 vdot 编码。调用环境须启用 VS，并确认目标硬件支持项目自定义指令；本次验证采用 Nexus-AM 裸机环境。

- 输入：有符号 INT8 行优先 A[8][32]，紧接 x[32]，共 288 字节。
- 输出：8 个 16 字节槽，共 128 字节。每槽低 32 位为 INT32 点积结果，高 96 位清零。槽步长为 16 字节，与完整向量存储一致。

- 两块缓冲区各自 16 字节对齐、有效且不重叠；输入不会被修改。调用者负责生命周期、数据打包及平台初始化。
- 沿用原 RV64 C ABI：栈及被调用者保存寄存器约束不变；向量寄存器和允许的调用者保存状态会改变。
- 返回状态为 vl=8、vtype=0（e8/m1/tu/mu）、vstart=0、vcsr=7，不保留原向量配置。

底层 vs1r.v 存储整个向量寄存器，输出行步长固定为 16 字节，因此接口要求 VLEN 恰好 128。VLEN256 的整寄存器存储会超出当前槽宽。强制 RVV 改变调用目标，完整静态库仍保留各函数，平台与缓冲区约定保持相同。

3.4.3 编译期选择与 RVV 回退

默认的 xs_c8_gemv_8x32 入口通过编译期别名连接配置复用 VDot 函数。设置 XS_C8_GEMV_FORCE_RVV=1 后连接相应 RVV 函数，显式 xs_c8_gemv_8x32_rvv 入口则始终选择 RVV。开关只接受字面量 0 或 1，错误配置在编译期被拒绝。

这一选择由头文件和匹配静态库共同形成。宏或 API 版本变化后，相关翻译单元一起重编，使调用点与库的选择一致。撤销 C8 时恢复原 firstdot 头文件和对应库即可回到原入口。编译期选择适用于当前明确的平台配置，调用方根据目标平台确认指令支持情况。

3.4.4 对象、装载镜像与调用表核对

比较对象依次为原合格 C8 镜像、原源码复制对照和 SDK 接入版。

表 3-6 对象、装载镜像与调用表核对

项目	结果
完整 BIN	三者全字节一致，均 31,440 字节
BIN SHA256	56805506eb98b68269c73a3645d8f75dff73b697e30dec5f93ccc4716089ad34
入口	均 0x80000000；ABI flags 及 ISA 属性一致
PT_LOAD	同地址、权限、对齐及完整内容；file30,024B，memory31,440B，零填充 1,416B
装载节	所有 SHF_ALLOC 节的位置、大小、权限、对齐及内容一致，包括 BSS
函数	16 个公共函数及启动/主程序等 5 个函数的地址、大小和机器码一致

表 3-6 对象、装载镜像与调用表核对（续）

项目	结果
测试表与数据	48 条 64 字节调用记录与原计划逐项匹配；12 组输入数组地址及内容一致
GNU 链接	无未解析符号；map 与实际子命令证明使用 SDK 库成员
完整 ELF 原字节	不同；差异仅在.debug_info、.debug_line、.debug_line_str
独立 GNU 复核	分别 objcopy --strip-debug 后，三份完整 ELF 全字节一致
去调试 ELF SHA256	7e26e295fb020a63334026f768ed4825372266465b4e6008973de966fefc04ab

调试节保存源文件/头文件路径及行号等信息，构建目录与头文件发生变化会改变这些记录。主检查器首先比较完整装载内容和调用计划，随后用去调试 ELF 作独立补充核对；三份原始 ELF 同时归档。

原 ELF SHA256：02159c7ef78cb31eb24cf706bf2974eadea313710563f235f2b11a7bb922de56。

复制对照 ELF SHA256：cfad14278ee4a3590de9bcbe7589b808fd91959156cef93ba92a78a90db0d99b。

SDK 接入 ELF SHA256：f0ddf1167c961101e70f58cfd012ae788d2965324ad057aa4ade5af1de5bc42e。

软件包接入按对象、链接和装载内容逐层核对。静态库只含完整 gemv.o，保留 16 个函数、原节和重定位；链接映射与 GNU 依赖文件确认调用程序实际使用 SDK 头和库成员。这样可以直接定位最终函数来自哪里，以及重定位和符号解析如何完成。

最终比较包含原合格镜像、原源码复制对照和 SDK 接入版。三者 BIN 均为 31440 字节且逐字节一致，入口为 0x80000000；PT_LOAD、BSS、函数地址与机器码、48 条调用表记录和 12 组输入数据也一致。装载内容相同，把接口改用 SDK 后的运行输入与原程序连接起来。

完整 ELF 的差异位于调试节。源文件路径、头文件位置和行号变化会改变这些信息，但装载段比较已经确认程序实际使用的字节相同。进一步用 GNU objcopy 去掉调试信息后，三份 ELF 也逐字节一致，形成与主检查不同路径的补充核对。

3.4.5 接入检查器的正负控制

7 项负控全部被拒绝：实际 ELF 代码字节改坏、调用表改坏、启动入口改变、BSS 大小改变、ISA 属性改变、输入计划改变，以及真实 GNU 链接错误空库。另有只改调试字节的正控，确认不会将纯调试变化判为行为改变。这些控制样例用于验证主机端镜像

检查器。

627 项输入及封存运行材料绑定 0f、EMU/NEMU 与 GNU 工具。原 33 个矩阵进程和 3 个 C 进程的记录关系及输出树保持，接入过程另行保存 GNU 构建和最终镜像等价回执，形成软件包与既有功能结果之间的连接。

成功流水线：2026-10-27T21:59:40.771873+08:00 至 2026-10-27T21:59:43.182911+08:00，exit0。两次 GNU 构建，8 份主流水线命令回执，另 3 份去调试 GNU 检查回执。流程首次通过，负控和原始文件全部保留。



图 3-7　C8 三方静态装载内容与调用输入对照

原合格夹具、同源重建与 SDK 实际链接版的 BIN、PT_LOAD 和有序调用表、输入身份一致。PT_LOAD 按 30024 字节文件内容与 1416 字节零填充的真实比例绘制；零填充包含对齐尾部，不能全部称为 BSS。调用及输入网格表示逻辑记录，不表示物理地址排列。调试节差异位于非装载区；627 个受保护文件已核验。静态等价支持沿用既有夹具证据，SDK 接入未新增 EMU、NEMU 或全核运行。

3.4.6　接入结果与功能记录复用

C8 包完成对象、可重定位链接和接回原 C 程序的完整镜像等价检查。原 16 函数、48 例两轮 96 次正常调用的功能记录可据此复用，其中 8×32 占 12 例、24 次调用。

该接入实验采用静态构建与镜像比较，EMU/NEMU 运行次数为 0。273 cycles 与

1.095238 倍来自原第 25 条性能矩阵，描述原 C8 完整调用；这里的结果说明 SDK 接入保持了原测试程序的装载内容和调用方式。

软件包已完成从普通 C 入口到最终镜像的接入验证。面向新应用时，可固定新的输入布局、调用路径和最终镜像，使用同路径 firstdot/RVV 比较完整调用成本。现有其他尺寸继续采用原选择：4×8 与 16×64 使用 firstdot，4×13 使用 RVV。

3.5 面积、时序与实现成本

3.5.1 评估条件与主要结果

使用 Nangate45 Typical 1.10V/25°C，Yosys0.9、ABC1.01 及 OpenSTA2.0.17。主档周期 1ns、负载 5fF，采用固定线负载模型，另有 0.5ns、2ns 及 1fF、20fF 敏感性档。

表 3-7 评估条件与主要结果

指标	FU	完整 VFEX4 父级
标准单元面积 μm²	17,298.246	284,614.946
nominal r2r 到达 ns	1.163404	3.261218
主档 i2r setup 裕量 ns	−0.268520	−2.393998

父级面积包含 FU 及 DIV。负裕量表明 1ns 参考目标仍有时序违例，整核 Fmax、功耗和布线后时序为待测。H1 相对 O2 独立 FU 面积降低 2.33386%，父级降低 0.137349%，同时保留部分路径退化。0f 的取消关钟修复以正确性为依据，其周期不退化与成本变化分别留档。

3.5.2 寄存状态与综合面积

面积采用 Nangate45 Typical 库，在相同映射流程和约束下统计标准单元面积。VDot FU 为 17298.246 平方微米，完整 VFEX4 父级为 284614.946 平方微米。父级包含 FU、DIV 和共享控制，因此父级数值描述集成成本，FU 数值用于观察点积单元自身的结构变化。

0f 相对修复前的父级面积减少 264.936 平方微米，约 0.0930%，FU 面积不变。父级 DFF 数量增加一个，回溯到既有整数 DIV 的 Bypass_final_rem_plus_d_reg[2] 在映射中的保留差异；两版对应 DIV 生成 RTL 相同。综合会跨逻辑进行优化和裁剪，因此面积变化由完整映射结果给出，并非简单按某段新增逻辑逐门相加。

H1 的 32 位保存优化与 0f 的门控修复分别记录。前者直接利用固定结果布局减少状态，后者改变父级取消后的清理控制。评估它们时，既查看局部模块，也查看共享 ready、门控及父级输出路径，能够观察局部优化在集成环境中的整体代价。

3.5.3 时序约束与路径分类

静态时序使用 1ns 周期、5fF 负载作为名义档，并配置理想时钟、0.05ns 时钟不确定度与 transition、0.1ns 输入输出 delay、0.05ns 输入 transition 和固定线负载模型。另设 0.5ns、2ns 以及 1fF、20fF 负载档，在同一映射网表上观察约束敏感性。

路径按输入、寄存器和输出分类，记为 i2r、r2r、r2o、i2o，同时单列 ready、门控使能及父级中的 VDot 子集。到达时间描述数据传播，要求时间描述路径预算，裕量是两者的差。比较时把路径类别、端点和约束放在一起，便于区分组合路径变化与外部预算变化。

名义档下，FU 与父级所报 i2r 路径的 setup 裕量分别为 −0.268520ns 和 −2.393998ns，当前映射结果尚未满足 1ns 目标。五档共 70 项类别与条件对照中，到达时间增大 30 项、减小 15 项、不变 25 项。条形图展示 0f 在 1ns / 5fF 名义档下的 14 类时序裕量，负裕量位于零线左侧。五档完整 70 项数值、修复前后的到达时间差及实际起止点保留在图数据附件与完整对照表。

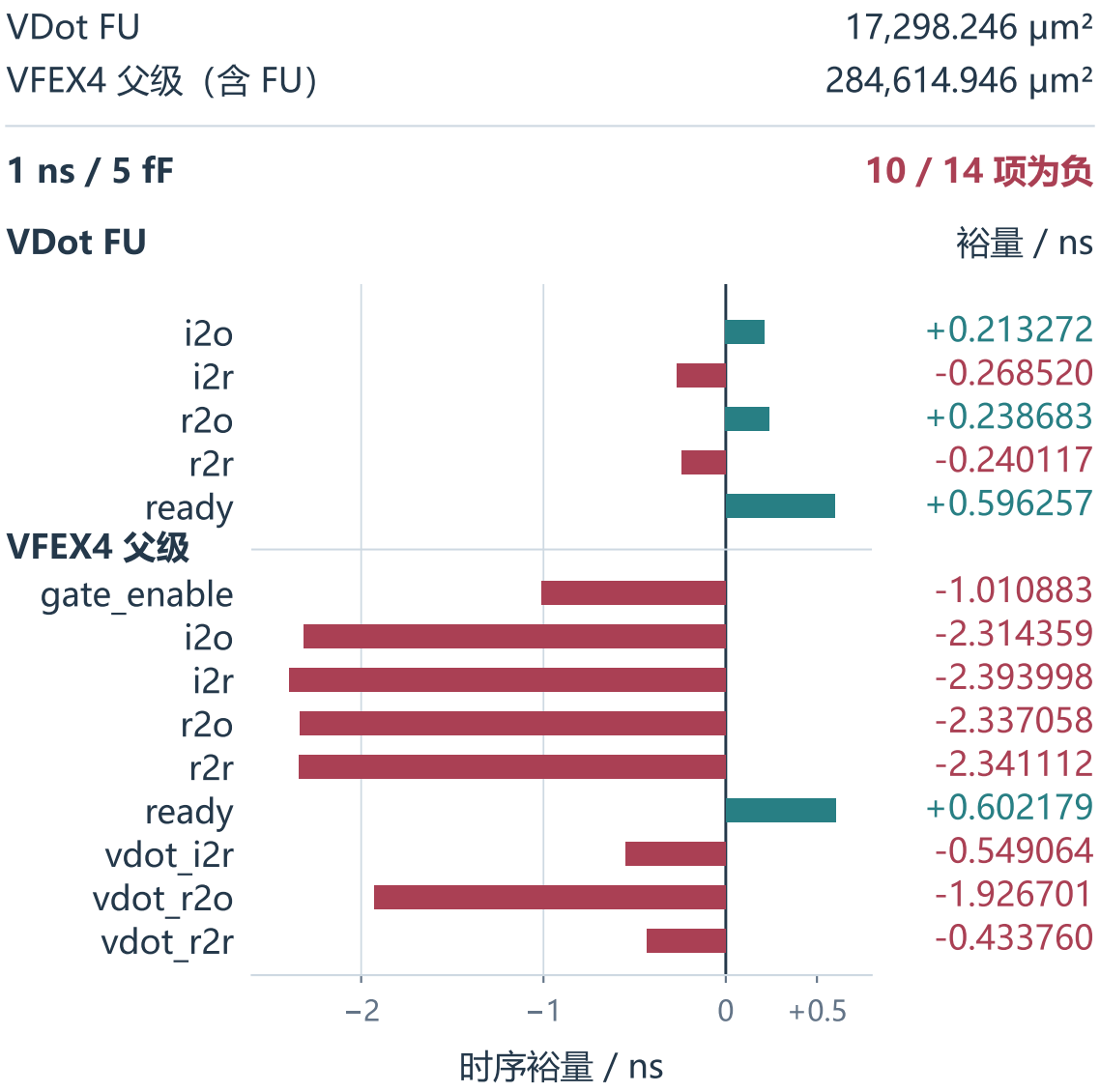


图 3-8　Nangate45 面积与时序裕量

Nangate45 Typical 1.10V、25°C 下的布局前评估。条形图展示 0f 在 1ns / 5fF 名义档的 14 类裕量，其中 10 类为负；五档 70 项完整数值保存在图数据附件。父级面积包含 FU。

3.5.4　父级关键路径与改进方向

0f 修复后的父级 ready 路径到达时间降低约 47.603ps，而 r2r、i2r 及 gate_enable 类别的最差到达分别增加约 94.304ps、119.132ps 和 32.965ps。部分路径改善与部分路径退化同时存在，说明完整父级中的控制、算术和门控网络对修改有不同响应。

端点回溯显示，两版 r2r 最差路径均位于 VFDiv，i2r 最差起点由 vsew 输入转为 src 输入，gate_enable 最差起点则在不同 VFDiv 实例间切换。类别中的最差路径可能随版本改变，因此优化时应检查实际起止点、逻辑深度与扇出，而不是仅根据类别名称判断同一条路径缩短了多少。

这些布局前结果给出了下一步时序工作的具体入口。可以优先分析完整父级中限制

名义档的路径，再以相同库和约束建立候选对照。布线寄生、多工艺角、整核 Fmax 和功耗属于后续物理评估内容；当前功能仿真、周期比较和静态时序分别描述已有工程的不同方面。

3.5.5 取消关钟与成本结果

DIV 取消后的清理需要经历明确的状态转移。父级识别匹配取消，子单元获得完成清理所需的边沿，随后撤销在途状态并回到空闲关钟。局部测试在 48 个取消窗口及其后 144 个采样拍中检查多余钟脉冲，观察目标是取消处理之后的时钟活动是否按预期停止。

门控输入使能与沿后采样的子时钟掩码代表不同位置的信号。使能参与 ClockGate 控制，掩码描述采样时哪些子时钟处于活动状态。结合占用、取消和清理事件阅读这些记录，能够解释为什么取消当拍仍需要一个有效清理边沿，以及何时可以重新接收其他功能单元的请求。

脉冲观察回答控制行为问题，面积和 STA 回答实现代价问题。动态功耗还取决于电容、电压、频率及各节点开关活动，需要相应模型与活动输入。当前评估据已有记录给出关钟行为、周期保持和同约束映射结果，后续功耗实验可以在相同功能场景上继续采集活动，保持比较对象一致。

3.5.6 门控修复的工程取舍

0f 解决了匹配 DIV 取消后的无效关钟保持。局部测试在 48 个取消窗口和其后 144 个采样拍中确认无多余钟脉冲，配套全核回归与周期比较进一步检查功能和执行代价。

修复的直接目标是恢复正确的取消与关钟行为。周期对照评估其执行代价，完整父级映射评估其面积与时序影响；动态功耗需要结合开关活动和功耗模型进一步测量。

最终工程组合采用 0f 修复：固定微基准周期保持一致，空闲关钟缺陷得到处理，父级面积小幅下降；名义档 r2r、i2r 及门控使能的到达时间分别增加约 94ps、119ps 和 33ps。后续时序优化可从这些实际路径入手，在相同库与约束下建立新候选对照。生产硬件对应提交为 0f12df675378556670afd0dd2153f47fcb330610。

3.6 验证汇总与设计结论

3.6.1 功能、复现与性能检查汇总

表 3-8 功能、复现与性能检查汇总

项目	验证结果	方法与结论
源码与输入身份	通过	0f 生产源码 398 项与 clean 副本及选定 Git 版本一致；完整工作树快照 24157 项覆盖源码和软件材料。
环境部署验证	通过，采用原记录	2026-10-09 hello/vadd 日志重新核对；阶段一 12 文件 SHA 及 220295598B 波形记录完成定位。
指令规格	内部合同已实现	SEW8/VLEN128，8 对 signed int8 点积，低 32 位结果、高 96 位清零；Q01–Q07 保留为待官方澄清项。
参考模型	主机检查通过	重新检查独立模型及确定性数据，覆盖 22 组定向与 10000 组随机输入。
功能单元与父级	原记录复算通过	0f 真实 FU 与 VFEX4 父级覆盖完整结果、反压、取消、身份、异常和严格 DIV 关钟；167 份原载荷及 4 ＋ 16 负控记录纳入复核。
整核协同仿真	组合验收通过	13 项原记录与 1 项新控制诊断，覆盖 117 个不同用例、124 次执行、O2 控制及负控恢复；宿主 UTC 问题按前述占用审计单列。
点积微基准	原周期记录复用	0f 修复前后 96 对正式窗周期一致；独立 batch128 为 221 对 678 cycles，加速比 3.067873 倍。
通用 GEMV 入口	输入复核与复现通过	四尺寸 768 窗口及 96 个普通 C 输出复现一致；33 个矩阵进程与 3 个普通 C 进程构成原运行集合，4×13 选择 RVV。
8×32 专用入口	最终镜像复核通过	627 项输入材料与原普通 C 镜像绑定，BIN、PT_LOAD、入口、48 表项和 12 数据组一致；原性能矩阵为 273 对 299 cycles。

表 3-8 功能、复现与性能检查汇总（续）

项目	验证结果	方法与结论
独立位宽扩展	独立单元验证通过	s16x8/r64 算术与包装扩展在原实验环境中复算；生产指令和 NEMU 采用 INT8。
面积与时序	成本复核通过，时序待收敛	0f 面积与五档 STA 的 1008 份文件回读；FU17298.246、父级 284614.946 平方微米，1ns 名义档存在负裕量。
独立构建复现	同机复现通过	独立源码、依赖、EMU/NEMU、AM/klib 和 25 个应用重建；复现使用同一主机，并复用工具及下载缓存。
entry-rollback	入口与回退明确	C8 静态库入口、RVV 回退、firstdot 原入口及 O1/B0 历史对照均保留。

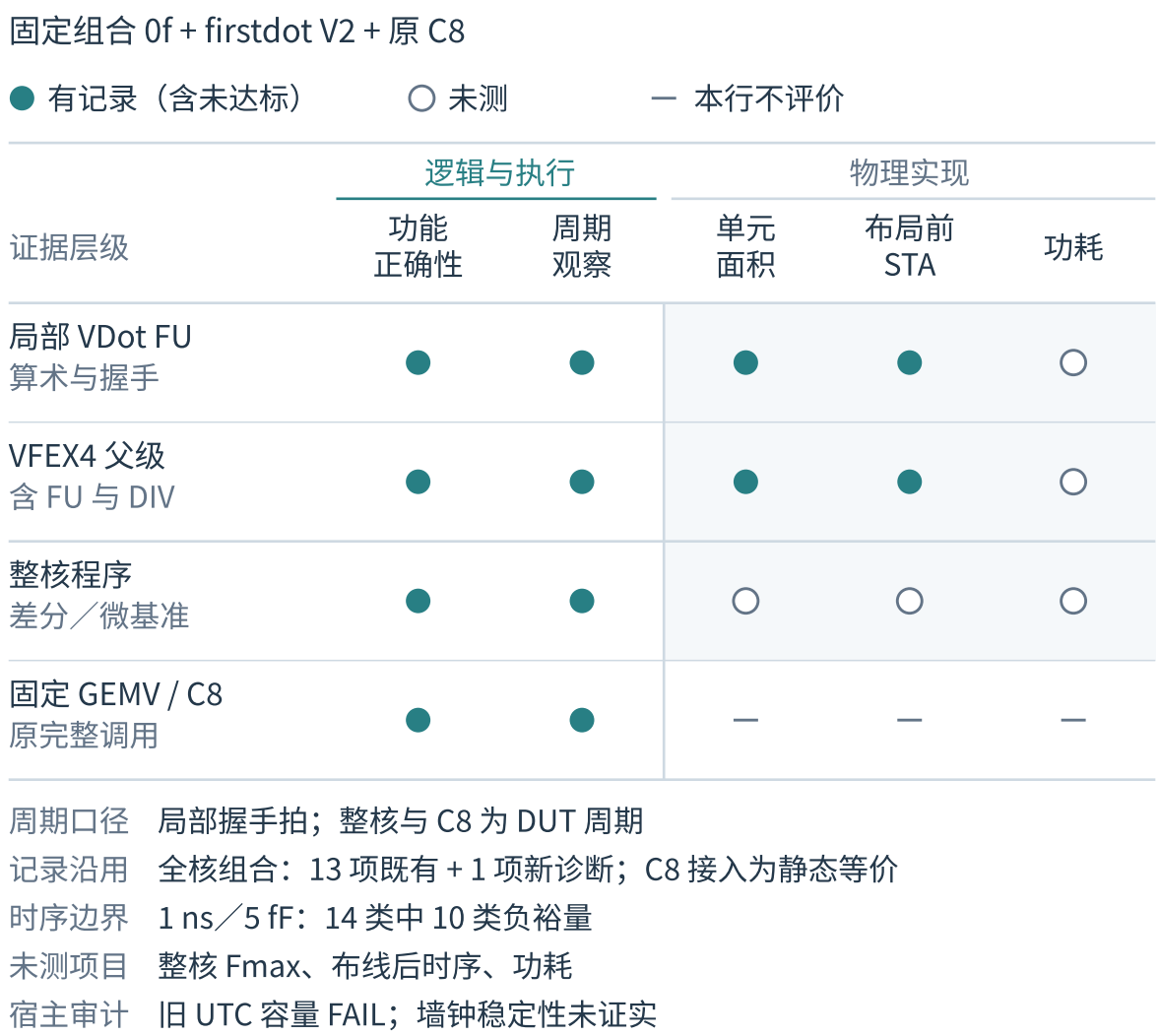


图 3-9 稳定组合的验收范围与保留边界

实心圆表示有相应记录，包含未达标结果，不等于 PASS；空心圆表示未测，横杠表示该软件证据行不评价物理实现。局部握手拍与整核、C8 的 DUT 周期分别记录。全核组合采用 13 项既有记录与 1 项新诊断，C8 接入为静态等价。旧 UTC 容量审计 FAIL、墙钟稳定性未证实均保留；1ns 名义档 14 类中 10 类负裕量，整核 Fmax、布线后时序和功耗未测。

3.6.2 优化候选的实测取舍

表 3-9 优化候选的实测取舍

探索	结果	最终处理
输入分离候选（P1/P1-R1）	对照窗口偏差超过预设 1% 限值	暂缓
单拍两槽候选（P4）	47 项成本检查中 6 项失败	未采用
单拍两槽修订（P4-R1）	47 项成本检查中 10 项失败	未采用

表 3-9 优化候选的实测取舍（续）

探索	结果	最终处理
C8 分段归约	实测 273.75→273 周期，一致性检查与 3% 收益目标未通过	未采用
FP32 到 QKV 探索（P3）	未取得完整计时窗口，尚未确认完整调用能否在 7200 秒预算内完成	暂缓

综合这些结果，工程选用 0f ＋ firstdot V2 ＋原 C8。应用评估包括点积微基准和固定 GEMV 调用；另有 INT16 独立单元扩展实验，生产指令保持 INT8。

3.6.3 阶段结论

本项目完成自定义指令硬件接入、独立 NEMU 扩展、分层功能验证及等价 RVV 周期分析。O2 提升独立吞吐，H1 降低结果状态成本，0f 处理 DIV 取消关钟，firstdot 与 C8 改善固定软件调用。固定源、镜像和样本支持结果复算，1ns 时序收敛及完整物理评估构成后续开发方向。

NEMU 补丁说明 • 测量合同 • 0f 性能与成本 • 版本数据对照

3.7 工程复现资料

3.7.1 固定版本与主要运行产物

测量软件以 T13 实现为基础，原文件及 SHA-256 保存在 team/references/zufan-t13/，对应提交为 f090665130107282be0990a7664aebf26e491b87。测量实验按自己的构建、镜像和运行记录建立样本集合。

该测量硬件源码提交为 4460300e9557ad20ac152fa67b1827c5d62802dd，工作负载的构建提交另行记录。

NEMU 以 a48a11f208ff81ad765591d83271d1b339fb3861 为基点，使用 vdot 补丁 1.1.0 及 xs-16ae9dd-vdot-runtime-tval-v1 平台配置。补丁绑定 7 个变化文件，SHA256 为 6c16 5bf5f94ef0bfb4f894a5f06eb6d620d4a7e0e12f3b1400fa72811f5d24db。

- 本章引用的历史资料发布提交为 eca41d7cd959f5f3c4f240e382919c492d1ccb96。
- 生产硬件提交为 0f12df675378556670afd0dd2153f47fcb330610。

- clean-02 EMU 的 SHA256 为 acf672eb39f836da4000b253849e3214089e9c8e5198060a57d2cc6bbd393da9。
- reference 的 SHA256 为 62b1707c52a4c323468481e0c1863b79f244d9a3fd54aeb7b7d2ecb5f01a35ca。
- C8 最终 BIN 的 SHA256 为 56805506eb98b68269c73a3645d8f75dff73b697e30dec5f93ccc4716089ad34。

以上标识对应本章采用的工程组合及主要运行产物，分项实验使用各自记录的构建与镜像身份。

3.7.2 两槽结构对照的样本记录

O1 与 O2 使用同一组 16 份冻结镜像，共执行 36 项（32 次首次镜像运行＋ 4 次代表 batch 镜像重放），478 个校验窗口，192 条正式性能窗口。每场景每模式每方法为 3 数据集 ×4measured ＝ 12 条样本；first/warmup 和重放完整保留，重放不增加独立样本数。周期降低按 $1 - \frac{\sum_i C_{\mathrm{O2,custom},i}}{\sum_i C_{\mathrm{O1,custom},i}}$，加速比按该模式本次实测的 $\frac{\sum_i C_{\mathrm{RVV},i}}{\sum_i C_{\mathrm{custom},i}}$。式中 C 为窗口周期数，i 为同一统计组内配对的正式窗口索引。每组逐 dataset/repeat/seq/镜像配对，机器码、工作量及退休指令一致；未扣循环/屏障或空窗开销。所有均值由该批正式窗口计算。single 对应完整调用，batch 对应四路同输入、独立目的的批量点积。

每个作业通过 DiffTest、完整 128 位输出、非 scratch 寄存器、CSR、哨兵和计数窗口检查，并满足各自 O1/O2 协议及有限排空条件。manifest 列出各内核使用的 scratch 集合，scratch_control 单独验证允许变化与受保护寄存器的划分。额外 4 次 batch 重放逐 seq 周期与退休数一致。30 项主机测试通过；全部原始 stdout/stderr、真实退出码、版本、输入和检查器摘要保留。

3.7.3 硬件对照与工具身份

表 3-10 硬件对照与工具身份

项目	O1	O2
实测 EMU SHA-256	6699225b76ceae479060741026683bb03de069c1e10a4a94e463304ba1e322d0	ac0123280bf6c742d27a7650505bc1d91db350d1519806edfd0be9c7ea208a57
构建源码提交	a239dace720b32216e83fb5a514930d334c69a26	b218cbc6e6b135e4ff219a902a9284a59d3c82c9
来源	从同一硬件输入重建	复用 G3 已验收固定 EMU

表 3-10 硬件对照与工具身份（续）

项目	O1	O2
O1_HANDOFF / O2_ELASTIC	1/0	0/1
O1_TRACE/O2_TRACE	1/0	0/1

429 项硬件源码与 21236 项依赖逐字节相同，MinimalConfig/单核/VLEN128/单线程 EMU；共同 FUSED_INPUT1、T12_TRACE0、T13_TRACE1。各自合法协议观察器只提供只读观察，共同 T13 提供全局周期/退休计数。

路径归一化后的实际 Verilator/C++ 命令、七类工具版本日志一致；实际 LLVM19.1.7 编译器由构建过程/proc 记录并核对。j2、12G JVM、O3/O0；最多两个单线程 EMU。新 O1 采用外置产品目录，原 G3 的 1687 份非对象生成文件已恢复并验证原始 SHA，旧 O2 EMU 不覆盖。构建记录分别列出硬件源码、实际工具和封存提交，便于对应实验输入与结果。

3.7.4 离线复算与模型数据

Windows 原生 Python 3.13.2 全量离线复算通过，覆盖 36 项作业、478 个窗口、192 条正式性能窗口及 4 项重放。见原生复算回执。复算直接读取封存的作业和窗口记录，Linux 构建来源与 Windows 复算回执相互对应。

交付时补齐了原冻结功能生成器依赖的两份模型语料，逐字节取自运行工具提交 63b95a5 并绑定 Git blob 及 SHA-256；原镜像、测量日志和 646 份主封存 payload 均未改变。原 30 项主机测试及新增 3 项语料绑定检查共 33 项通过；首次封装复算失败和修复后成功记录均保留。Windows 请用 Python UTF-8 模式执行：

```
python -X utf8 -B performance/portable.py --archive evidence/20261019-performance-ab --campaign campaign-01 --output /absolute/new/portable-verification.json
```

上述相对路径适用于 Windows 交付目录；仓库内对应 team/optimizations/o2 下的路径。

3.7.5 取消关钟修复的对照输入

表 3-11 取消关钟修复的对照输入

对象	修复前	修复后
实现	O2 两槽＋ H1 低 32 位保存	O2 ＋ H1 ＋ DIV 取消关钟修复

表 3-11 取消关钟修复的对照输入（续）

对象	修复前	修复后
硬件源码	8b5024fcbd96986871cc40ce8c9fb71a67c495eb	0f12df675378556670afd0dd2153f47fcb330610
EMU SHA256	e37b3c267cae29036bda0b6952a17db2523ca0b3a573d45281341d4207df8747	2d1db77cc407486a2387a4a69da2498b68d44980239025eebcc02370c02826e8

硬件 429 份 Git 源码、21236 份依赖、每版 1672 份生成产物、实际编译命令和七类工具版本均经核验。生产源码仅 ExeUnit.scala 不同。

同一 MinimalConfig/单核单线程/O2 与 T13 观察配置、128MB、seed=1、正常 NEMU 和 flash；208 份冻结软件载荷不变。

正常 NEMU SHA256：efbed0774addc27eac38af1a19ab5d9c9dd5f179b5015f25ae7b80b966185bd4。

生成物差异包括 Git 标识、只读诊断日志、顶层注释及标量 FDiv 公共子表达式内联；这些差异与生产源码变化一起纳入生成物审查。

3.7.6 修复前后逐窗周期核对

表 3-12 修复前后逐窗周期核对

场景	修复前 vdot 均值	修复后 vdot 均值	两版 RVV 均值	两版 RVV/vdot
单次点积 1	71.000	71.000	85.250	1.200704×
依赖链 256	1857.000	1857.000	5440.500	2.929725×
四路独立批量 128	221.000	221.000	678.000	3.067873×
加载/计算/存储 64	1127.500	1127.500	1133.000	1.004878×

单位均为 DUT 全局 cycles。每场景三组数据 × 四轮，vdot 和 RVV 分别配对；192 正式窗口形成 96 对，每对差值均为 0，退休指令与工作量相同。

共 478 个校验窗口，保留 first/warmup；四次额外 batch 重放完全一致，不加入正式样本。36 项实际 EMU 均通过完整 128 位结果、DiffTest、GOOD TRAP、窗口计数和 O2 生命周期/排空检查。

各场景加速比按 $\frac{\sum_i C_{\mathrm{RVV},i}}{\sum_i C_{\mathrm{vdot},i}}$ 计算，其中 C 为窗口周期数，i 为该场景中配对的正式窗

口索引；保留共同计时开销。修复前后逐窗周期一致，表明 0f 在这组固定微基准中保持了原有性能。

详见逐窗差值、全部样本、复算回执。

3.7.7 综合面积与完整接口

修复前采用 H1 封存的 205 份生成、综合和 STA 文件，经逐字节核验后作为基线；修复后重新生成 RTL 并完成综合与 STA，形成同条件对照。

两边使用相同完整生产接口、相同未修改 O4 脚本、Yosys0.9/ABC1.01/OpenSTA2.0.17 及其动态库、Nangate45 typical 25°C/1.1V 库。

ABC 目标 1000ps、BUF_X1 驱动、5fF 映射负载。面积是 Liberty 单元面积之和，单位 μm²，非芯片布局面积。

表 3-13 综合面积与完整接口

边界	修复前面积	修复后面积	增量	增量比例	DFF 数量	ICG 数量
fu	17298.246	17298.246	+0.000	+0.000000%	1742→1742	0→0
parent	284879.882	284614.946	-264.936	-0.092999%	17171→17172	3→3

父级映射 DFF 增加 1 个，已回溯到既有整数 DIV 的 Bypass_final_rem_plus_d_reg[2]：旧网表该位无驱动和消费者，新网表保留 DFF_X1。两版对应 DIV 生成 RTL 相同，没有新增 RTL 寄存器声明或 VDot 状态位；该差异来自整体综合优化与裁剪，面积因此按完整映射结果统计。详情在归档 dff-review.json。

两版 VDot 均保存 32 位结果，并在公开 128 位输出处补齐高 96 位零；算术与控制/perf 寄存器分组保持一致。父级 DIV 使用自己的完整结果格式。

FU 生成 5 份文件完全相同。父级 58 份生成文件仅 ExeUnit.sv 和 LogPerfEndpoint.sv 不同；后者只改变打印顺序，并被实际 SYNTHESIS 定义排除。

3.7.8 五档时序详细结果

同一映射的五档条件为 1ns/5fF、0.5ns/5fF、2ns/5fF、1ns/1fF、1ns/20fF。理想时钟、0.05ns uncertainty 与 transition、0.1ns 输入/输出 delay、0.05ns 输入 transition、5K_hvratio_1_1 线负载。parent testEnable=0，不额外排除 ready/reset/flush 路径。

每版 70 条类别/档位路径对照，父级网表规范化经逐 bit 拓扑绑定；未知单元和动态未约束端点拒收。以下 nominal 到达时间包含相应外部输入延迟；表格按路径类别比较，各版实际最差端点另行列出。

表 3-14 五档时序详细结果

边界/路径	修复前 arrival(ns)	修复后 arrival(ns)	差值 (ps)	修复后 slack(ns)
fu/i2o	0.636728	0.636728	+0.000	0.213272
fu/i2r	1.190323	1.190323	+0.000	-0.268520
fu/r2o	0.611317	0.611317	+0.000	0.238683
fu/r2r	1.163404	1.163404	+0.000	-0.240117
fu/ready	0.253743	0.253743	+0.000	0.596257
parent/gate_enable	1.855397	1.888362	+32.965	-1.010883
parent/i2o	3.128214	3.164359	+36.145	-2.314359
parent/i2r	3.194971	3.314103	+119.132	-2.393998
parent/r2o	3.185769	3.187058	+1.289	-2.337058
parent/r2r	3.166914	3.261218	+94.304	-2.341112
parent/ready	0.295424	0.247821	-47.603	0.602179
parent/vdot_i2r	1.511395	1.474949	-36.446	-0.549064
parent/vdot_r2o	2.832115	2.776701	-55.414	-1.926701
parent/vdot_r2r	1.345889	1.356106	+10.217	-0.433760

70 条类别/档位比较中，到达时间变大 30 条、变小 15 条、相同 25 条。全部差值和端点见完整时序 CSV。

nominal 所报路径中，修复前有 10 类负 slack，修复后有 10 类。两版在 1ns 目标下均未收敛；整核 Fmax、布线后时序与功耗列为后续评估内容。

最差端点回溯已保存 path-endpoint-review.json：前后 r2r 均是 VFDiv 的 fsm_q[0] 到 quo_m1_iter_q_f64_0[1]，i2r 最差起点从 vsew 输入变为 src 输入；gate_enable 最差起点从 vfdivs_1 切到 vfdivs_0，终点仍是浮点 DIV 的 ICG。这些端点属于完整父级的时序分析对象，端点变化与到达时间变化共同用于解释取舍。

3.7.9 复算入口与实验归档

主机检查共 62 项，包括周期配对 30 项、验收身份 6 项、成本审计 20 项和归档拒错 6 项，均通过。预检中曾发现验收 JSON 字段适配和生成范围遗漏日志模块两项问题，修订后完成复核；对应失败回执与成功结果一并保存。

原始周期根：/home/riscv/riscv-lab/o2-r3-div-gate-performance-20261025。

原始综合根：/home/riscv/riscv-lab/o2-r3-div-gate-cost-20261025。

原 H1 完整 build：/home/riscv/riscv-lab/o2-h1-build-before-r3-div-gate-20261024。

原始 stdout/stderr、真实退出码、命令、源/依赖/镜像/工具/库哈希及所有路径报告随归档保存；EMU/NEMU 和 EDA 可执行文件保留在本地，用 SHA 绑定，不放进日常 Git 提交。

周期和成本复算入口分别为 performance/recheck.py 与 cost/compare.py，使用独立输出路径保存新回执。package_evaluation.py --verify ARCHIVE --receipt NEW_JSON 用于核对归档字节。完整归档保存源、工具、库、镜像和路径报告的对应关系，可据此复算统计或重新安排相同输入的实验。

3.7.10 C 接口与编译开关

表 3-15 C 接口与编译开关

入口	目标
xs_c8_gemv_8x32，开关缺省或 0	config_reuse_gemv_8x32_reuse_vdot
xs_c8_gemv_8x32，XS_C8_GEMV_FORCE_RVV=1	config_reuse_gemv_8x32_reuse_rvv
xs_c8_gemv_8x32_rvv	始终 config_reuse_gemv_8x32_reuse_rvv

XS_C8_GEMV_FORCE_RVV 最终展开只能是字面量 0 或 1；空值、表达式和未知标识不受支持，编译时拒绝。独立 guard 为 XS_C8_SELECTED_GEMV_H，API 版本 XS_C8_GEMV_API_VERSION=1。XS_C8_GEMV_PROFILE 为静态配置标签，CPU 能力由调用环境确认。修改宏或 API 版本后，相关翻译单元一同重编，以保持调用点与库选择一致。

C8 使用独立的开关、guard 和入口别名，与原 XS_GEMV_FORCE_RVV 及 xs_gemv_* 接口并存。应用选择 C8 或 firstdot 时，分别使用对应的头文件与静态库。

3.7.11 对象构建与接入命令

独立 GNU 汇编参数固定为 -march=rv64gcv -mabi=lp64d -mno-relax -c。完整 gemv.o 为 27,160 字节，SHA256 为 f8bc13daeead98a414f313600a5778c96ac95a66b5bcb5c714ea3b98e0d25ab3。确定性 libxs_gemv_c8.a 只含一个 gemv.o，保留 16 个函数、原节和重定位；config_reuse_gemv.h 从合格 gemv.h 原字节复制，仅改产物文件名。

软件包绑定源基点 c823a8786e052c6e26f6915563c42b6d730dd8e3、选中 RTL 0f12df

675378556670afd0dd2153f47fcb330610 和 clean-02 配套身份。完整输入、工具身份及验收结果以 INPUTS.json、验证器输出和版本数据档案记录为准。验收同时检查对象内容、链接关系与运行输入身份。

包级静态检查覆盖完整对象、单库成员、缺省及 0/1 配置下两别名的 GNU 调用对象、可重定位链接等价、符号解析、错误配置拒绝和 SDK 搬移。example.c 提供编译与链接示例。随后的小节给出接回原 C 程序的最终镜像等价结果，将包级检查与原矩阵、普通 C 调用的功能记录衔接起来。

接入新应用时，根据 wrapper、分派、CSR 保存、对象组合和最终布局的变化安排目标运行；性能比较同时计入调用者实际承担的工作。叶函数字节核对用于确认计算主体，完整镜像和调用计划则用于确认最终运行方式。

本包内设 XS_C8_GEMV_FORCE_RVV=1 或显式调用 xs_c8_gemv_8x32_rvv 后完整重编。撤销 C8 选用时，恢复原 firstdot 头和匹配库并完整重编。C8 稳定别名对应 8×32，其余尺寸沿用各自选定的 firstdot 或 RVV 入口。

3.7.12 链接对象与检查入口

- 原主程序只把底层头的包含改为 selected_gemv_c8.h。
- 8×32 reuse vdot/RVV × 三数据集，共 6 个函数表项改用 xs_c8_gemv_8x32 / xs_c8_gemv_8x32_rvv。其余 42 项、数据、期望值、顺序和校验不变，测试向量沿用原计划。
- 真正从 libxs_gemv_c8.a(gemv.o) 链入完整合格对象；SDK 程序源目录没有 gemv.S，对象目录没有绕过库的 gemv.o。GNU 依赖文件和 link map 确认实际包含 SDK 头、实际抽取库成员。
- 在全新输出中分别构建原源码复制对照和 SDK 接入程序，保留原 AM/klib 和链接次序；只有输出路径和记录链接来源的选项不同。

接入仅调整头文件与六个 8×32 函数表项，其余调用计划和数据保持一致。

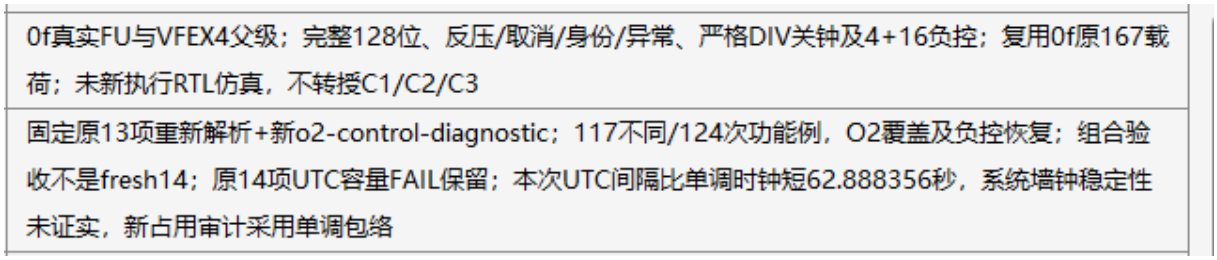
3.7.13 宿主运行记录与并发审计

原 14 项全核记录的 UTC 容量检查出现 80.119ms 表观四路重叠，状态为 FAIL/WALL_CLOCK_LIMITED。后续按既定合同重新执行 o2-control-diagnostic，其余 13 项从原始日志重新解析，形成组合验收。

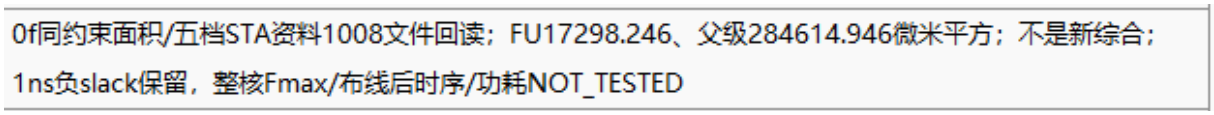
最终组合 14 项通过功能、NEMU 差分、O2 覆盖和负控后同镜像恢复。**组合采用 13 项复用记录与 1 项新执行记录**。选定记录的严格 UTC 并发峰值为 3（上限 3）；新单进程以同 boot/PID 的单调时钟包络验证，峰值 1，包络 1497.000 秒。该耗时是宿主审计数据，不用于 DUT 加速比。

新控制诊断检查 131072 字节寄存器结果，退出 0；独立离线审计再次通过。原 13 项的占用记录采用 UTC，新控制诊断另有单调时钟包络。

本次 UTC 间隔与单调包络仍相差-62.888356 秒（UTC 更短）。因此墙钟稳定性仍未证实；新占用审计依赖同 boot 的单调包络，而非容差修正 UTC。原 UTC 容量失败及两种时间基准的差异均保存在审计记录中。



（a）FU、父级与全核验证记录



（b）面积与时序记录

图 3-10　R5 功能与成本验收摘要

功能条目记录局部与全核验证的采用情况；成本条目记录面积结果和 1ns 目标下的负裕量。

3.7.14　工程使用与回退资料

当前 1ns STA 尚未收敛，功耗、整核 Fmax 和布线后时序待评估。INT16 扩展已有独立单元结果，生产全核继续采用 INT8。4×8 和 16×64 的配置复用控制差异保留为后续分析方向，4×13 采用 RVV。内部规格 Q01–Q07 仍待官方澄清。

选定的 0f ＋ firstdot V2 ＋原 C8 作为稳定工程组合，保留可使用入口与回退路径。后续候选围绕父级关键时序、软件调用配置及实际应用工作量开展比较，沿用已经明确的输入、镜像和样本组织。

冻结文件 24157 项，生产源码 398 项与 0f 及 clean 副本一致。快照在 /home/riscv/riscv-lab/software-import-20261026/r5-g5-acceptance-01/frozen-workspace.tar.gz，已逐项回读，含工作树文件但不含 Git 历史和外部依赖。使用/回退见 README，逐项身份见数据记录，原始新增证据见 manifest。失败尝试与原 28 条数据档案共同保存在工程记录中。

3.7.15　技术资料索引

周期测量边界与统计合同；O1 与 O2 同镜像验证过程；0f 硬件性能与五档成本详细记录；C8 调用 ABI、对象及回退合同；C8 最终链接和镜像等价验证；R5 组合验收和证据复用。